1. Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?

(i) Kill processes by name
(ii) Kill a process based on the process name
(iii) Kill a single process at a time with the given process ID

CODE:

```
~
  GNU nano 8.7
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
    }
    else if (pid == 0) {
        // Child process
        printf("Child Process\n");
        printf("PID  : %d\n", getpid());
        printf("PPID : %d\n", getppid());
    }
    else {
        // Parent process
        printf("Parent Process\n");
        printf("PID  : %d\n", getpid());
        printf("Child PID : %d\n", pid);
        wait(NULL);
    }
    return 0;
}
```

Output:

```
Ansh@Anshs-Rog MSYS ~
$ gcc fork.c -o fork

Ansh@Anshs-Rog MSYS ~
$ ./fork
Parent Process
PID  : 525
Child PID : 526
Child Process
PID  : 526
PPID : 525

Ansh@Anshs-Rog MSYS ~
$ 5~
```

2.Write a program for process creation using C

(iv) Orphan Process

(v) Zombine Process

CODE:

```
  GNU nano 8.7
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(5);
        printf("Child Process\n");
        printf("PID  : %d\n", getpid());
        printf("PPID : %d\n", getppid());
    }
    else {
        printf("Parent exiting\n");
    }
    return 0;
}
```

OUTPUT:

```
~

Ansh@Anshs-Rog MSYS ~
$ nano orphan.c

Ansh@Anshs-Rog MSYS ~
$ gcc orphan.c -o orphan

Ansh@Anshs-Rog MSYS ~
$ ./orphan
Parent Process
Child Process
PID  : 534
Child PID : 535
PID  : 535
PPID : 534

Ansh@Anshs-Rog MSYS ~
$
```

2. Create the process using fork () system call.

(i) Child Process creation
(ii) Parent process creation
(iii) PPID and PID

CODE:

```
~
  GNU nano 8.7
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child exiting\n");
    }
    else {
        sleep(10);
        printf("Parent running\n");
    }
    return 0;
}
```

Output:

```
Ansh@Anshs-Rog MSYS ~
$ nano zombie.c

Ansh@Anshs-Rog MSYS ~
$ gcc zombie.c -o zombie

Ansh@Anshs-Rog MSYS ~
$ ./ zombie
-bash: ./: Is a directory

Ansh@Anshs-Rog MSYS ~
$ zombie.c
-bash: zombie.c: command not found

Ansh@Anshs-Rog MSYS ~
$ ./zombie
Child exiting
Parent running

Ansh@Anshs-Rog MSYS ~
$ ps -el
      PID    PPID    PGID     WINPID   TTY         UID    STIME COMMAND
      499     498     499       4716  pty0      197609 16:35:32 /usr/bin/bash
      498       1     498      23764  ?         197609 16:35:32 /usr/bin/mintty
      557     499     557       5088  pty0      197609 16:47:08 /usr/bin/ps

Ansh@Anshs-Rog MSYS ~
$
```